



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



Proyecto Final:

RSA Y TOY BLOCK CIPHER



INTRODUCCIÓN A LA CRIPTOGRAFÍA

Profesora || **Sandra Díaz Santiago**

Grupo || **6CV4**

Alumno || **Areli Alejandra Guevara Badillo**

05 de enero de 2026

1. ENVOLTURA DE CLAVES (KEY WRAPPING) CON RSA

Diseñe e implemente un programa informático para cifrar una clave para su cifrador por bloques de juguete (toy block cipher), utilizando RSA.

INCISO 1

Generación del par de claves pública (e, n) y privada (d, n).

Se detalla cada una de las funciones implementadas para la generación de las claves para RSA:

```
1 # Funcion que genera las llaves RSA
2 def generar_llaves(bits):
3     p, q = parPrimos(bits)
4     n = p * q
5     phi = (p - 1) * (q - 1)
6     e = genE(phi)
7     d = genD(e, phi)
8     return e, d, n
```

generar_llaves(bits): Es la función principal del módulo que coordina el flujo de generación del par de claves. Su función es instanciar la creación de los primos, calcular el módulo n , ϕ , y obtener los exponentes.

Devuelve una tupla con el exponente público (e), el exponente privado (d) y el módulo (n).

```
1 # Funcion que genera un numero primo
2 def genPrimo (bits):
3     return number.getPrime(bits)
```

genPrimo(bits): Utiliza bibliotecas criptográficas para obtener un número que tenga una alta probabilidad de ser primo, basándose en la longitud de bits especificada.

Devuelve un número entero primo de la longitud de bits solicitada.

```
1 # Funcion que genera un par de primos distintos
2 def parPrimos (bits):
3     while True:
4         p = genPrimo(bits)
5         q = genPrimo(bits)
6         if p != q:
7             return p, q
```

parPrimos(bits): Su propósito es garantizar la obtención de dos factores primos, p y q , que sean obligatoriamente distintos entre sí. Esto evita vulnerabilidades donde n sea un cuadrado perfecto.

Devuelve un par de números primos diferentes (p, q).

Proyecto Final: RSA y Toy Block Cipher

```
1 # Funcion que genera el exponente publico e
2 def genE (phi):
3     while True:
4         e = random.randrange(2, phi)
5         while number.GCD(e, phi) == 1
6             and e != 3 and e != 65537:
7             if e % 2 != 0:
8                 return e
9             else:
10                e += 1
```

genE(phi): Esta función selecciona el exponente público e. Realiza una búsqueda aleatoria de un número que sea coprimo con phi y verifica que no sea un valor trivial, asegurando que sea impar para optimizar procesos matemáticos.

Devuelve un número entero e que cumple con las condiciones de coprimidad necesarias para el cifrado RSA.

```
1 # Funcion que genera el exponente privado d
2 def genD (e, phi):
3     return number.inverse(e, phi)
```

genD(e, phi): Implementa el cálculo del exponente privado d mediante el algoritmo extendido de Euclides para encontrar el inverso multiplicativo modular de e respecto a phi.

Devuelve un número entero d, el cual permite realizar la operación de descifrado.

Funcionamiento:

The screenshot shows a Python IDE with two files open: 'publica' and 'privada'. The 'publica' file contains the public key: 8627943566342515783 8845844909121281377. The 'privada' file contains the private key: 2384970687665015447 8845844909121281377. The terminal window shows the execution of the 'keysRSA.py' script. The script prompts the user to select an option (1 for key generation, 2 for encryption, 3 for decryption). Option 1 is selected, and the script prompts for the number of bits (32), the name for the public key ('publica'), and the name for the private key ('privada'). The keys are generated and saved. The terminal also shows the command to run the script: `"C:/Users/Areli Guevara/AppData/Local/Programs/Python/Python314/python.exe" "c:/Users/Areli Guevara/OneDrive - Instituto Politecnico Nacional/Documents/Escuela/Introduccion a Criptografia/proyFinal/src/keysRSA.py"`.

```
PS C:\Users\Areli Guevara\OneDrive - Instituto Politecnico Nacional\Documents\Escuela\Introduccion a Criptografia\proyFinal\src> & "C:/Users/Areli Guevara/AppData/Local/Programs/Python/Python314/python.exe" "c:/Users/Areli Guevara/OneDrive - Instituto Politecnico Nacional/Documents/Escuela/Introduccion a Criptografia/proyFinal/src/keysRSA.py"
1. Generar mis llaves RSA (Pares de llaves)
2. Alice: Generar y cifrar clave K de 32 bits
3. Bob: Descifrar clave K recibida
Seleccione una opcion:1

Ingrese el numero de bits para RSA: 32
Nombre para guardar tu CLAVE PÚBLICA: publica
Nombre para guardar tu CLAVE PRIVADA: privada
Llaves generadas y guardadas.

Presiona Enter para continuar...
Presione una tecla para continuar . . .
```

INCISO 2

Cifrado de una clave K de 32 bits haciendo uso de RSA.

Se detalla la funcione para el cifrado RSA:

```
1 # Funcion que cifra el mensaje r
2 def cifrarMensaje (r, e, n):
3     return pow(r, e, n)
```

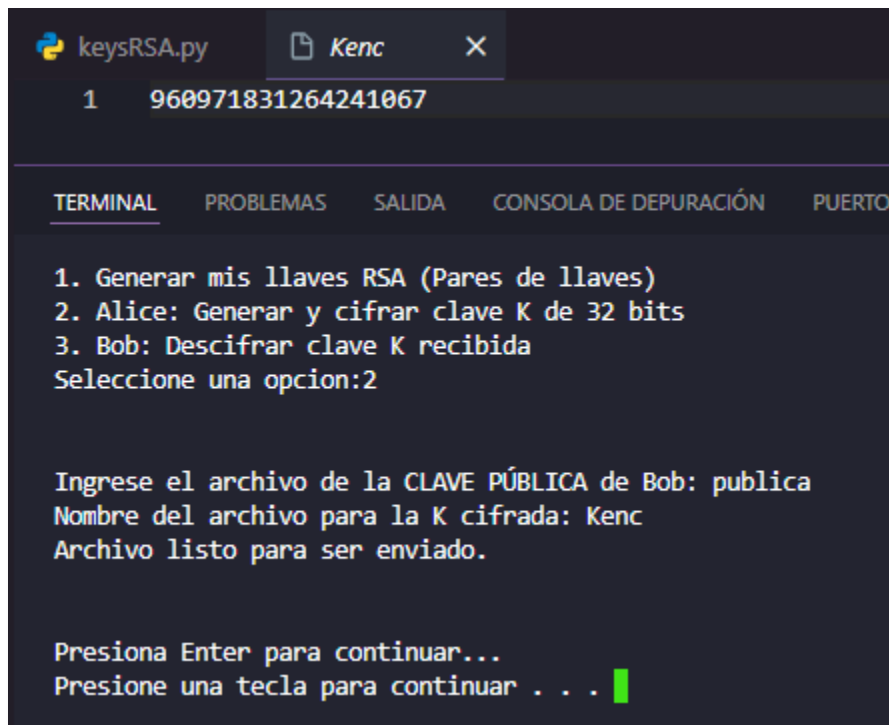
Esta función recibe el mensaje y aplica una **exponenciación modular** utilizando los componentes de la clave pública del destinatario.

La implementación utiliza la función `pow(base, exp, mod)` de Python, la cual está optimizada para manejar números de gran tamaño de manera eficiente, evitando desbordamientos de memoria y reduciendo el tiempo de cómputo.

Calcula la operación matemática $C = r^e \bmod n$.

Devuelve un número entero que representa la clave cifrada.

Funcionamiento



The screenshot shows a Python IDE with two tabs: 'keysRSA.py' and 'Kenc'. The 'keysRSA.py' tab is active, displaying a single line of code: `1 960971831264241067`. Below the code editor is a terminal window with the following text:

```
1. Generar mis llaves RSA (Pares de llaves)
2. Alice: Generar y cifrar clave K de 32 bits
3. Bob: Descifrar clave K recibida
Seleccione una opcion:2

Ingrese el archivo de la CLAVE PÚBLICA de Bob: publica
Nombre del archivo para la K cifrada: Kenc
Archivo listo para ser enviado.

Presiona Enter para continuar...
Presione una tecla para continuar . . .
```

INCISO 3

Descifrado de la clave K usando RSA.

Se detalla la funcione para el descifrado RSA:

```
1 # Funcion que descifra el mensaje cifrado
2 def descifrarMensaje (cifrado, d, n):
3     return pow(cifrado, d, n)
```

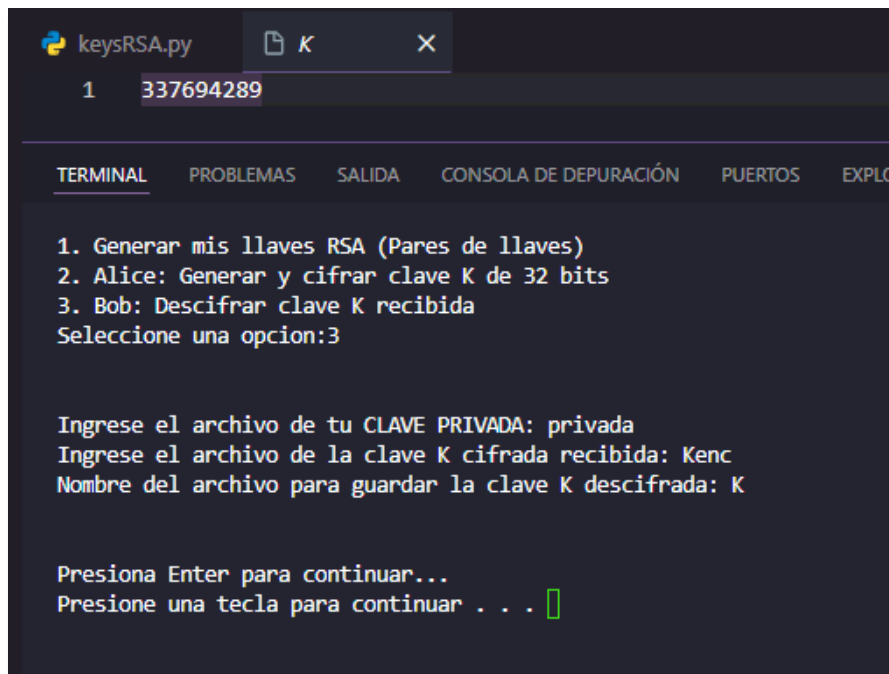
Esta función recibe el mensaje cifrado y aplica una **exponenciación modular** utilizando los componentes de la clave privada del destinatario.

La implementación utiliza la función `pow(cifrado, d, mod)` de Python, la cual está optimizada para manejar números de gran tamaño de manera eficiente, evitando desbordamientos de memoria y reduciendo el tiempo de cómputo.

Calcula la operación matemática $M = c^d \bmod n$.

Devuelve un número entero que representa la clave descifrada.

Funcionamiento



The screenshot shows a code editor window titled 'keysRSA.py' with a single line of code: `1 337694289`. Below the editor is a terminal window with the following text:

```
1. Generar mis llaves RSA (Pares de llaves)
2. Alice: Generar y cifrar clave K de 32 bits
3. Bob: Descifrar clave K recibida
Seleccione una opcion:3

Ingrese el archivo de tu CLAVE PRIVADA: privada
Ingrese el archivo de la clave K cifrada recibida: Kenc
Nombre del archivo para guardar la clave K descifrada: K

Presiona Enter para continuar...
Presione una tecla para continuar . . .
```

2. CIFRADOR DE BLOQUE DE JUGUETE Y MODO CONTADOR

INCISO 1

Utilizando sus programas de sesiones anteriores, genere una Sbox S , la inversa de la sbox S^{-1} , una permutación P y la permutación inversa P^{-1} .

A continuación, se coloca la implementación de las funciones que generan las sustituciones, permutaciones y sus inversas:

```
1 // Función para generar la función S
2 void funcionS(unsigned char s[16]) {
3     for(int i = 0; i < 16; i++) {
4         s[i] = i;
5     }
6
7     srand(time(NULL));
8     for(int i = 15; i > 0; i--) {
9         int j = rand() % (i + 1);
10        unsigned char temp = s[i];
11        s[i] = s[j];
12        s[j] = temp;
13    }
14 }
```

Esta función genera una **S-box (Caja de Sustitución)** de 4 bits (16 valores posibles). El algoritmo inicializa un arreglo con valores secuenciales del 0 al 15 y posteriormente aplica el algoritmo de intercambio para desordenarlos de manera pseudoaleatoria.

```
1 // Función para generar la inversa de la función S
2 void inverseS(unsigned char s[16], unsigned char invs[16]) {
3     for(int i = 0; i < 16; i++) {
4         invs[s[i]] = i;
5     }
6 }
```

Se encarga de calcular la **S-box inversa S^{-1}** . Recorre la caja S generada previamente y asigna el índice original al valor correspondiente en la nueva tabla.

```
1 // Función para generar una permutación
2 void permutacion(int p[8]) {
3     for(int i = 0; i < 8; i++) {
4         p[i] = i;
5     }
6     for(int i = 7; i > 0; i--) {
7         int j = rand() % (i + 1);
8         int temp = p[i];
9         p[i] = p[j];
10        p[j] = temp;
11    }
12 }
13 }
```

Su objetivo es generar una **permutación P** de 8 elementos. Similar a la función de la S-box, utiliza una semilla aleatoria para reorganizar las posiciones de los elementos en un bloque.

Proyecto Final: RSA y Toy Block Cipher

```
1 // Función para generar la permutación inversa
2 void invPermutacion(int p[8], int invP[8]) {
3     for(int i = 0; i < 8; i++) {
4         invP[p[i]] = i;
5     }
6 }
```

Calcula la **permutación inversa** P^{-1} necesaria para el proceso de descifrado. Mapea cada posición de destino de vuelta a su posición de origen.

Funcionamiento:

The screenshot shows a VS Code editor with two files open: 's' and 'toy.c'. Both files contain a table with 8 rows and 2 columns. The first column contains a letter or symbol, and the second column contains a sequence of 8 characters. The first file 's' has a header 'S' and a footer 'S^-1(z)'. The second file 'toy.c' has a header 'n' and a footer 'P^-1'. Below the files is a terminal window showing the execution of 'toy.exe'. The terminal output includes a menu with three options: '1. Generar permutaciones y sustituciones', '2. Cifrar', and '3. Descifrar'. It prompts the user to 'Elige una opcion: 1'. Then it prompts for the file name for 'S' and 'P', and finally shows the message 'Archivos generados exitosamente. Presiona Enter para continuar...Presione una tecla para continuar . . .'. The terminal window also has tabs for 'TERMINAL', 'PROBLEMAS', 'SALIDA', 'CONSOLA DE DEPURACIÓN', 'PUERTOS', and 'EXPLORADOR'.

INCISO 2

Modificar la implementación del cifrado/descifrado de bloque de juguete para que trabaje en modo contador.

En este apartado se incluyen las modificaciones para que el cifrado Toy funcione en modo contador y maneje archivos mas grandes, se plantean los siguientes puntos a considerar:

- Se cifra el archivo por bloques de 8 bits, por lo que el IV es de 8 bits.
- Las funciones para aplicar la sustitución y permutación se tomaron de anteriores practicas por lo que se omitirá su explicación.
- El IV se genera al momento de cifrar y se guarda en un archivo para el descifrado.
- Gracias al modo contador el cifrado y el descifrado hacen uso de las mismas funciones, solo cambiando los archivos que se le envían.

Proyecto Final: RSA y Toy Block Cipher

Explicación de las funciones:

```
1 // Función que cifra/descifrar un bloque de 8 bits
2 unsigned char cifrarBloque(unsigned char iv, unsigned char k[4], unsigned char S[16], int P[8]) {
3     unsigned char M = iv;
4     for (int i = 0; i < 3; i++) {
5         M ^= k[i];
6         M = aplicarSustitucion(M, S);
7         M = aplicarPermutacion(M, P);
8     }
9     M ^= k[3];
10    return M;
11 }
```

Esta función representa el núcleo del cifrador de bloque de juguete. Implementa una estructura de **3 rondas** de cifrado donde, en cada iteración, se aplica una operación XOR con una sub-clave, seguida de una sustitución no lineal (S-box) y una difusión por permutación (P-box). Finalmente, se aplica una última mezcla con la cuarta sub-clave.

Transforma el valor actual del contador (de 8 bits) en un byte pseudoaleatorio utilizando la clave de 32 bits proporcionada.

Devuelve un byte de *keystream* que será utilizado para la operación XOR con el texto plano.

```
1 // Función para procesar un archivo en modo CTR
2 void procesarArchivoCTR(const char* nombreEntrada, const char* nombreSalida,
3 unsigned char k[4], unsigned char S[16], int P[8], unsigned char iv) {
4     FILE *fIn = fopen(nombreEntrada, "rb");
5     FILE *fOut = fopen(nombreSalida, "wb");
6
7     if (!fIn || !fOut) {
8         printf("Error al abrir los archivos.\n");
9         return;
10    }
11
12    unsigned char byteEntrada, byteSalida, keystream;
13    unsigned char contador = iv;
14
15    while (fread(&byteEntrada, 1, 1, fIn) == 1) {
16        keystream = cifrarBloque(contador, k, S, P);
17        byteSalida = byteEntrada ^ keystream;
18        fwrite(&byteSalida, 1, 1, fOut);
19        contador++;
20    }
21
22    fclose(fIn);
23    fclose(fOut);
24 }
```

Proyecto Final: RSA y Toy Block Cipher

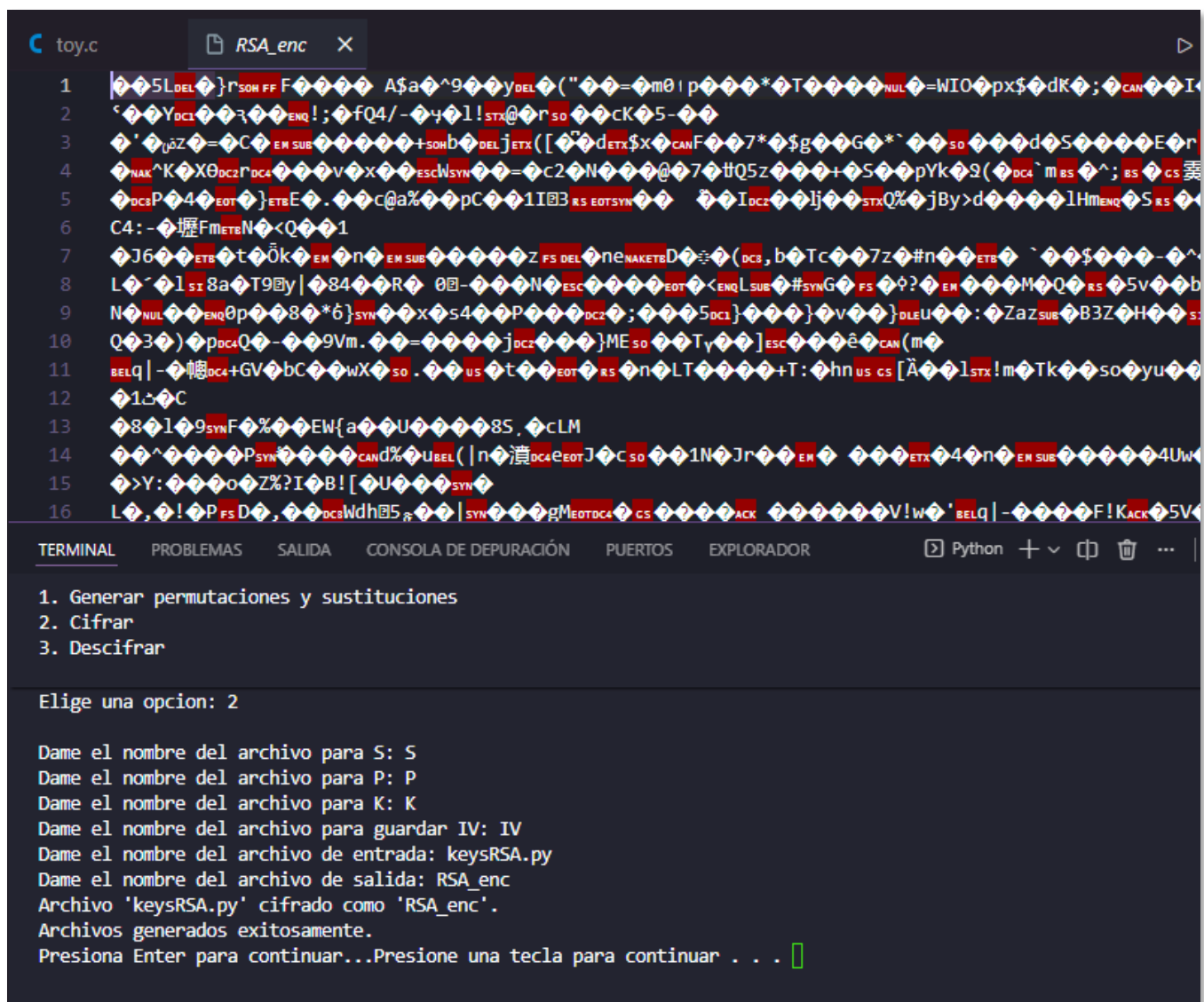
Es la función encargada de la gestión de archivos y la implementación del modo de operación. Su diseño permite procesar archivos de cualquier tamaño byte por byte.

Para cada byte del archivo original, la función cifra el valor actual del **contador** para generar un byte de flujo, realiza la operación XOR con el mensaje y, posteriormente, incrementa el contador para el siguiente bloque.

Devuelve la generación de un archivo de salida que contiene el texto cifrado (si la entrada es texto plano) o el texto recuperado (si la entrada es un texto cifrado), cumpliendo con la propiedad de simetría del modo CTR donde se utiliza el mismo procedimiento para cifrar y descifrar.

Funcionamiento:

Cifrado: Vamos a cifrar el código fuente del punto 1.



```
toy.c RSA_enc X
1  A$a^9yDEL("=m0!p*T?NUL=WIOpx$dK;CAN?I
2  ^YDC1?3?ENQ!;fQ4/-4?1!STX@rso?cK5-?
3  ?'jZ?=C?EM SUB?+SOHb?DELjETX([?dETX$X?CANF?7*?g?G?*`?SO?d?S?E?n
4  ?NAX^K?X0DC2?DC4?V?X?ESCWSYN?=c2?N?@?7?#Q5Z?+?S?pYk?9(DC4`mBS^;BS?CS
5  ?DC3P?4?EOT?}ETBE?c@a%?pC?1I@3RS EOTSYN? ?I?DC2?J?STXQ%?jBy>d?lHmENQ?SRS?
6  C4:-?FMEtEN?<Q?1
7  ?J6?ETB?t?k?EM?n?EM SUB?ZFS DEL?neNAKETED?DC3,b?Tc?7z?#n?ETB`?`$??-?^
8  L?^?l?si8a?T9y|?84?R?0@-?N?ESC?EOT?ENQLSUB?#SYNG?FS?P?EM?M?Q?RS?5v?b
9  N?NUL?ENQ?0p?8?6}?SYN?x?s4?P?DC2?;?5DC1}?v?}DLEU?:ZaZSUB?B3Z?H?S:
10 Q?3?)?pDC4Q?-?9Vm.?=?jDC2?}ME?SO?Ty?}ESC?ê?CAN(m?
11 BELq|-?DC4+GV?bC?wX?SO.?US?t?EOT?RS?n?LT?+T:hnUSCS[Ä?1STX!mTk?so?yu?
12 ?1?C
13 ?8?l?9SYNF?%?EW{a?U?8S.?cLM
14 ?^?P?SYN?CAND%?UBEL(|n?DC4EOTJ?cSO?1N?Jr?EM? ?ETX?4?n?EM SUB?4Uw
15 ?>Y:??o?Z%?I?B![?U?SYN?
16 L,?!?PFS D,?DC3Wdh@5?|SYN?gMEOTDC4?CS?ACK?V!w?BELq|-?F!KACK?5V

TERMINAL PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN PUERTOS EXPLORADOR Python + v [ ] ...

1. Generar permutaciones y sustituciones
2. Cifrar
3. Descifrar

Elige una opción: 2

Dame el nombre del archivo para S: S
Dame el nombre del archivo para P: P
Dame el nombre del archivo para K: K
Dame el nombre del archivo para guardar IV: IV
Dame el nombre del archivo de entrada: keysRSA.py
Dame el nombre del archivo de salida: RSA_enc
Archivo 'keysRSA.py' cifrado como 'RSA_enc'.
Archivos generados exitosamente.
Presiona Enter para continuar...Presione una tecla para continuar . . .
```

Descifrado:

```
toy.c RSA.py X
1 from Crypto.Util import number
2 import random
3 import os
4
5 path_llaves = "datos/llaves/"
6 path_key = "datos/keys/"
7 path_cifrado = "datos/cifrado/"
8 path_mensajes = "datos/mensajes/"
9
10 # Funcion que genera las llaves RSA
11 def generar_llaves(bits):
12     p, q = parPrimos(bits)
13     n = p * q
14     phi = (p - 1) * (q - 1)
15     e = genE(phi)
16     d = genD(e, phi)
17     return e, d, n
18
19 # Funcion que genera un numero primo
20 def genPrimo (bits):
21     return number.getPrime(bits)
22
23 # Funcion que genera un par de primos distintos
24 def parPrimos (bits):
25     while True:
26         p = genPrimo(bits)
27         q = genPrimo(bits)
28         if p != q:
29             return p, q
30
31 # Funcion que genera un archivo de texto
32 def generar_archivo_texto(nombre_archivo, contenido):
33     with open(nombre_archivo, 'w') as f:
34         f.write(contenido)
35
36 # Funcion que genera un archivo de texto
37 def generar_archivo_binario(nombre_archivo, contenido):
38     with open(nombre_archivo, 'wb') as f:
39         f.write(contenido)
40
41 # Funcion que genera un archivo de texto
42 def generar_archivo_json(nombre_archivo, contenido):
43     with open(nombre_archivo, 'w') as f:
44         f.write(contenido)
45
46 # Funcion que genera un archivo de texto
47 def generar_archivo_xml(nombre_archivo, contenido):
48     with open(nombre_archivo, 'w') as f:
49         f.write(contenido)
50
51 # Funcion que genera un archivo de texto
52 def generar_archivo_yaml(nombre_archivo, contenido):
53     with open(nombre_archivo, 'w') as f:
54         f.write(contenido)
55
56 # Funcion que genera un archivo de texto
57 def generar_archivo_csv(nombre_archivo, contenido):
58     with open(nombre_archivo, 'w') as f:
59         f.write(contenido)
60
61 # Funcion que genera un archivo de texto
62 def generar_archivo_tsv(nombre_archivo, contenido):
63     with open(nombre_archivo, 'w') as f:
64         f.write(contenido)
65
66 # Funcion que genera un archivo de texto
67 def generar_archivo_ods(nombre_archivo, contenido):
68     with open(nombre_archivo, 'w') as f:
69         f.write(contenido)
70
71 # Funcion que genera un archivo de texto
72 def generar_archivo_xls(nombre_archivo, contenido):
73     with open(nombre_archivo, 'w') as f:
74         f.write(contenido)
75
76 # Funcion que genera un archivo de texto
77 def generar_archivo_xlsx(nombre_archivo, contenido):
78     with open(nombre_archivo, 'w') as f:
79         f.write(contenido)
80
81 # Funcion que genera un archivo de texto
82 def generar_archivo_xlsb(nombre_archivo, contenido):
83     with open(nombre_archivo, 'w') as f:
84         f.write(contenido)
85
86 # Funcion que genera un archivo de texto
87 def generar_archivo_xlsm(nombre_archivo, contenido):
88     with open(nombre_archivo, 'w') as f:
89         f.write(contenido)
90
91 # Funcion que genera un archivo de texto
92 def generar_archivo_xltx(nombre_archivo, contenido):
93     with open(nombre_archivo, 'w') as f:
94         f.write(contenido)
95
96 # Funcion que genera un archivo de texto
97 def generar_archivo_xlt(nombre_archivo, contenido):
98     with open(nombre_archivo, 'w') as f:
99         f.write(contenido)
100
101 # Funcion que genera un archivo de texto
102 def generar_archivo_xlws(nombre_archivo, contenido):
103     with open(nombre_archivo, 'w') as f:
104         f.write(contenido)
105
106 # Funcion que genera un archivo de texto
107 def generar_archivo_xlwb(nombre_archivo, contenido):
108     with open(nombre_archivo, 'w') as f:
109         f.write(contenido)
110
111 # Funcion que genera un archivo de texto
112 def generar_archivo_xltx(nombre_archivo, contenido):
113     with open(nombre_archivo, 'w') as f:
114         f.write(contenido)
115
116 # Funcion que genera un archivo de texto
117 def generar_archivo_xlt(nombre_archivo, contenido):
118     with open(nombre_archivo, 'w') as f:
119         f.write(contenido)
120
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812

```